Practical No. : 1 Date : / /

Aim: Write a short note on Software Projects Vs other types of Projects.

Ans :

The biggest difference between software and the products of other kinds of projects is it's not physical. Software consists of ideas, designs, instructions and formulas. Creating software is almost entirely a cognitive activity. The stuff we can see and measure, from code files They stand in for the real stuff, vs. the other way around. Still, software only matters when it appears as something real, even as barely real as coloured squiggles on a computer screen. Maintaining that connection from thought stuff to real stuff is one of software's peculiar challenges. In comparing the management of different projects, the first trap that many people fall into is that they do not differentiate between managing the technology and managing the project process. Certainly, there is a lot of difference between managing how concrete is placed and how the interface in a software development is placed on a computer screen.
Many techniques in general project management also apply to software project management, but Fred Brooks identified some characteristics of software projects which make them particularly difficult :-

Invisibility : When a physical artifact such as a bridge is constructed the progress can actually be seen. with software, progress is not immediately visible. Software project management can be the process of making the invisible visible.

Complexity : Per dollar, pound or euro spent, software products contain more complexity than other engineered artifacts.

Conformity : The 'traditional' engineer usually works with physical systems and materials like cement and steel. These physical systems have complexity, but are governed by consistent physical laws. Software developers have to conform to the requirement of human clients. It is not just that individuals can be inconsistent. Organizations, because of lapses in collective memory, in internal communication or in effective decision making, can exhibit remarkable, ‘organizational stupidity'.

Flexibility : That software is easy to change is seen as a strength. However, where the software system interfaces with a physical or organizational system, it is accommodating the other components rather than vice versa. Thus, software systems are particularly subject to change.

Feature	Software Project	Other Project
Tangible	Not Tangible	It is tangible.
End Product	Not clearly defined	Very clearly defined
Production	No fixed production plan, difficult to monitor and track	Fixed production plan which can be tracked
Productivity	Productivity varies greatly with change in technology or worked	Productivity does not vary much
Project Methodology	Varies widely based on project	Typically, standard
Management methodology	Managing a software project is more managing interpersonal communication and less administration.	It is more about maintaining schedule and good administration
Transfer of Ownership	The transfer of software is tricky as the organisation doesn't own the hardware which runs the software	Transfer is easy as the company own the project till it hands over
Multitasking	Difficult to multitask the resources	Production resources can be used for multiple projects
Personalisation	It is very easy to change product as per customer requirement at any time.	Can be personalized to a certain extent, but difficult in the middle of the project
Leadership	Software Projects need leaders and managers, not just administrators.	A capable administrator is enough to run an ordinary project.

Marks

Content	Total

Signature

Practical No. : 2 Date : / /

Aim: Prepare Functional and non-functional requirements of term project.

Ans :

Project Name: Credit card fraud detection system.

Problem Statement: An online system to monitor fraudulent activities on credit card transactions.

Functional Requirements :

User:

- Login into the system
- Logout of the system
- View About Section
- Upload Credit Data Files
- View Uploaded Files
- View predictions of future fraudulent transactions based on values entered for features v1,v2,v3…...
- Enter form data manually for prediction.
- Upload single CSV record of a customer containing all credit card transactions with their feature values.
  (Due to confidentiality issues, the CSV records does not contain names of features but instead the are named as v1, v2, v3…..)
- Upload multiple CSV records.
- Delete Uploaded Files.
- Analyse Uploaded Files Data.
- View Files Report.

Non–Functional Requirements:

Performance Requirements:

- The application must be interactive.
- Each page must load within 2 seconds.
- The system must be scalable enough to support 10,000 visits at the same time while maintaining optimal performance.

Safety Requirements

- The data in the database can be lost or can be corrupted due some failure or error conditions. Hence a periodic backup of both the database and the source code should be taken.
- The mean time to restore the system (MTTRS) following a system failure must not be greater than 10 minutes.

Security Requirements

- The software should have proper authentication so that it can support various roles of the users that the software supports.
- Also, proper authorization is required so that a user can only access and modify the data that he is supposed to.

Software Quality Attributes

- Maintainability: The software should be maintainable by the admin and he should be able to manage the software bugs.
- Usability: The software should satisfy the needs of maximum number of users, project managers and developers. The system must perform without failure in 95 percent of use cases during a month.
- The web dashboard must be available to users 99.98 percent of the time every month during business hours.

Marks

Content	Total

Signature

Practical No. : 3 Date : / /

Aim: Estimate the project cost using COCOMO.

Ans :

In modern software development practice, it is crucial to know the cost and time required for the software development before establishing new software projects. One of the efficient cost estimation models which are extensively applied to many software projects is called "Constructive Cost Model (COCOMO)".
Software projects under COCOMO model strategies are classified to 3 categories including, organic, semi-detached, and embedded.

Organic:

This suits a small software team since it has a generally stable development environment.
The problem is well understood and has been solved in the past.

Semi-detached:

The software project which requires more experience and better guidance and creativity.
For example, Compiler or different Embedded System

Embedded:

The project with operating tight constraints and requirements is under this category.
The developer requires high experiences and has to be creative to develop complex models.

	Organic	Semi-detached	Embedded
Project size (lines of source code)	2,000 to 50,000	50,000 to 300,000	300,000 and above
Team Size	Small	Medium	Large
Developer Experience	Experienced developers needed	Mix of Newbie and experienced developers	Good experience developers
Environment	- Familiar Environment	- Less familiar environment	- Unfamiliar environment (new) - Coupled with complex hardware
Innovation	Minor	Medium	Major
Deadline	Not tight	Medium	Very tight
Example(s)	Simple Inventory Management system	New Operating system	Air traffic control system

Types of COCOMO model:

COCOMO model allows software manager to decide how detailed they would like to conduct the cost estimation for their own project. COCOMO can unveil the efforts and schedule of a software product based on the size of the software in different levels.

There are basic COCOMO, Intermediate COCOMO, and Detailed/Completed COCOMO.

1. **Basic COCOMO model**

   The basic COCOMO is used for rough calculation which limits accuracy in calculating software estimation. This is because the model solely considers based on lines of source code together with constant values obtained from software project types rather than other factors which have major influences to Software development process as a whole.

2. **Intermediate COCOMO model**

   Intermediate COCOMO model is an extension of the Basic COCOMO model which includes a set of cost drivers into account in order to enhance more accuracy to the cost estimation model as a result.

3. **Complete/detailed COCOMO model**

   The model incorporates all qualities of both Basic COCOMO and Intermediate COCOMO strategies on each software engineering process. The model accounts for the influence of the individual development phase (analysis, design, etc.) of the project.

Calculation

In COCOMO, the general calculation steps of COCOMO-based cost estimation are the following:

Get an initial estimate of the development effort from evaluation of thousands of delivered lines of source code (KDLOC).

Determine a set of 15 cost factors from various attributes of the project.

Calculate the effort estimate by multiplying the initial estimate with all the multiplying factors i.e., multiply the values in previous steps.

Effort applied to the project: $E = a_b(KLOC)^{b_b}$ (in Person-month)
Development time: $D = c_b(E)^{d_b}$ (in month)
Manpower required: $P = \frac{E}{D}$ (in Person)
Where a_b, b_b, c_b, d_b are constants for each category of software product

SW project	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semi detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

In our Project, we have 30 KLOC , so applying basic COCOMO Model :

Basic Mode →	$E = a_b(KLOC)^{b_b} = 2.4\ (30)^{1.05} =\ 85.35\ PM$ $D = c_b(E)^{d_b} = 2.5\ (85.35)^{0.38} =\ 13.54\ months$ $P = \frac{E}{D} = \frac{85.35}{13.54} = 6\ people$
Semi Detached →	$E = a_b(KLOC)^{b_b} = 3.0\ (30)^{1.12} =\ 135.36\ PM$ $D = c_b(E)^{d_b} = 2.5\ (135.36)^{0.35} =\ 13.9\ months$ $P = \frac{E}{D} = \frac{135.36}{13.9} = 10\ people$
Embedded →	$E = a_b(KLOC)^{b_b} = 3.6\ (30)^{1.20} =\ 213.22\ PM$ $D = c_b(E)^{d_b} = 2.5\ (213.22)^{0.32} =\ 5.561\ months$ $P = \frac{E}{D} = \frac{213.22}{5.561} = 38\ people$

Marks

Analysis	Formula / Calculation	Total

Signature